

Sample 2

SPRING 2026

Instructor: Dr. Rahul Bhadani

Deep Learning for Engineering Applications

Examination Sample 2

April 30, 2026 • 3:00 PM - 5:30 PM • 100 Points

⚠ Important Instructions

- **Permitted:** One two-sided handwritten cheat sheets (letter size or smaller), calculator, pen/pencil, eraser
- **Prohibited:** textbooks, electronic devices, collaboration of any kind
- Read every question carefully; do not assume prior familiarity with phrasing
- Use all provided space; request extra sheets if needed
- Write clearly – illegible work receives **0 pts**
- **Circle** or **box** every final answer
- Show all work for partial credit on numerical sections

Name: _____

A#: _____

Total Pages: 31 (including supplementary materials) • *By signing below, you affirm adherence to the academic integrity policy.*

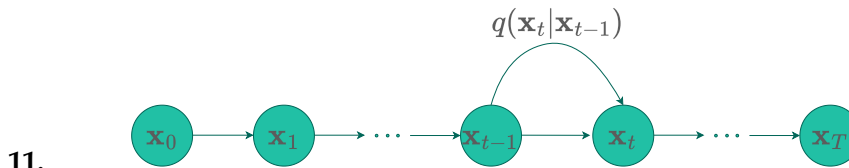
Signature: _____

Section 1 – ☒ – True / False

20 Points

For each statement, circle **T** (True) or **F** (False). Each question is worth **1 pts**. No partial credit.

1. The discriminator's goal is to generate samples that look like they came from $p_{data}(x)$
☐ T ☐ F
2. In fractionally-strided convolution, we upsample the input.
☐ T ☐ F
3. Minimizing the GAN objective function with an optimal discriminator is equivalent to minimizing the JS-divergence.
☐ T ☐ F
4. In a denoising AutoEncoder, we predict a noisy version of dataset from a clean dataset.
☐ T ☐ F
5. KL divergence is asymmetric.
☐ T ☐ F
6. In Variational Inference, we are not looking one single value but a best function.
☐ T ☐ F
7. The mean-field variational family assumes that the latent variables are independent of each other in the approximate posterior.
☐ T ☐ F
8. Denoising Autoencoder lets you create unlimited synthetic dataset based on distribution of the training dataset.
☐ T ☐ F
9. Automatic Differentiation framework can handle random variable but we use reparametrization trick to ease computation.
☐ T ☐ F
10. Isotropic Gaussian are symmetric in all direction.
☐ T ☐ F



The forward process of Diffusion model is shown. Based on the framework of forward process discussed, $q(\mathbf{x}_t | \mathbf{x}_{t-1})$ is fully known.

☐ T ☐ F

12. In Denoising Diffusion Probabilistic Model, a suitable choice of loss function is KL-divergence.

☐ T ☐ F

13. In ODE flow map, the trajectory line can intersect each other.

☐ T ☐ F

14. If a system represented by differential equations that doesn't depend on independent variable (e.g. time) in the right-hand side, then the system is called autonomous system.

☐ T ☐ F

15. A limit cycle is an isolated closed orbit in the phase plane of dynamical system.

☐ T ☐ F

16. Brownian motion is so irregular that it has infinite variation.

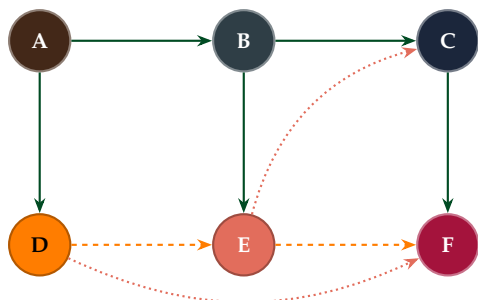
☐ T ☐ F

17. Flow matching is to ODE what score matching is to SDE.

☐ T ☐ F

18. Markov property states that the current state depends on the future states as well as the past state.

☐ T ☐ F



19.

The given graph is an example of directed graph.

☐ T ☐ F

20. The value function specifies how good it is for an agent to be in a particular state.

☐ T ☐ F

21. A normalizing flow always maps a simple base distribution to uniform distribution.

☐ T ☐ F

22. The ELBO is equal to the negative KL divergence up to an additive constant.

☐ T ☐ F

Section 2 – – Multiple Choice

20 Points

Circle the single best answer. 2 pts each. No partial credit.

Q1. In a Neural ODE, the function $f(\mathbf{z}(t), \theta(t))$ is typically parameterized by:

- (A) A lookup table.
- (B) A neural network with parameters θ
- (C) A polynomial approximation.
- (D) A Fourier series.

Q2. In a Neural ODE, the output $\mathbf{y}$ is obtained by:

- (A) Applying $F(\mathbf{x})$ directly
- (B) Taking the mean of all hidden states
- (C) Solving $\mathbf{z}(t_1)$ with initial condition $\mathbf{z}(0) = \mathbf{x}$ where t_1 is the query time (final time in the integration)
- (D) Applying the final layer weight matrix.

Q3. Neural ODEs achieve $\mathcal{O}(1)$ memory during training because:

- (A) The adjoint method recomputes the forward state during the backward pass
- (B) They use quantized weights
- (C) The network has fewer parameters
- (D) They store all intermediate activations

Q4. Why are Neural ODEs particularly well-suited for irregularly-sampled time series?

- (A) They model continuous-time dynamics, so observations can occur at arbitrary times
- (B) They require data to be on a uniform grid
- (C) They ignore time information entirely
- (D) They interpolate missing values with nearest-neighbor

Q5. Consider a code-snippet with a graph definition using PyTorch Geometric

```

1  import torch
2  from torch_geometric.data import Data
3
4  x = torch.tensor([
5      [1.0, 0.0, 1.0],
6      [0.5, 1.0, 0.0],
7      [0.0, 1.0, 1.0],
8      [1.0, 0.5, 0.5],
9      [0.5, 0.0, 1.0],
10     [0.0, 0.5, 0.0],
11 ], dtype=torch.float)
12
13
14  edge_index = torch.tensor([
15     [0, 0, 0, 1, 1, 2, 2, 3, 4], # sources
16     [1, 2, 3, 3, 4, 4, 5, 5, 5], # targets
17 ], dtype=torch.long)
18
19
20  edge_attr = torch.tensor([
21     [0.9], [0.7], [0.5],
22     [0.8], [0.6],
23     [0.4], [1.0],
24     [0.3], [0.2],
25 ], dtype=torch.float)
26
27  y = torch.tensor([0,1,0,1,0,1], dtype=torch.long)
28
29  data = Data(x=x, edge_index=edge_index, edge_attr=edge_attr, y=y)

```

Which one is true for the given graph?

- (A) Each edge has two features.
- (B) There are 6 nodes in the graph and there is an edge from node 2 to node 5.
- (C) There is a self-loop in the graph.
- (D) `data.x[2]` is `tensor([1.0, 0.5, 0.5])`

Q6. Adjacency matrix tells us about

- (A) how many features an edge has in a graph.
- (B) how nodes are connected to each other in a graph.
- (C) how many features a node has.
- (D) whether a graph is planar or not.

Q7. In the context of message-passing graph neural network, which one is the node pdate

rule:

- (A) $\mathbf{h}_u^{(k+1)} = \text{UPDATE}^{(k)}\left(\mathbf{h}_u^{(k)}, \text{AGGREGATE}^{(k)}(\{\mathbf{h}_v^{(k)}, \forall v \in \mathcal{N}(u)\})\right)$
- (B) $\mathbf{h}_u^{(k+1)} = \text{AGGREGATE}^{(k)}\left(\mathbf{h}_u^{(k)}, \text{UPDATE}^{(k)}(\{\mathbf{h}_v^{(k)}, \forall v \in \mathcal{N}(u)\})\right)$
- (C) $\mathbf{h}_u^{(k+1)} = \text{UPDATE}^{(k)}\left(\mathbf{h}_u^{(k)}, \text{UPDATE}^{(k)}(\{\mathbf{h}_v^{(k)}, \forall v \in \mathcal{N}(u)\})\right)$
- (D) $\mathbf{h}_u^{(k+1)} = \text{AGGREGATE}^{(k)}\left(\mathbf{h}_u^{(k)}, \text{AGGREGATE}^{(k)}(\{\mathbf{h}_v^{(k)}, \forall v \in \mathcal{N}(u)\})\right)$

Q8. Variance Preserving method is useful for

- (A) Variational Autoencoder
- (B) Generative Adversarial Network
- (C) Neural Ordinary Differential Equations
- (D) Diffusion Model

Q9. In ODE interpretation of Diffusion Model, ODE is defined via a time-dependent vector field $u : [0, 1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d$. u_t is the target vector field at time t . If we are to approximate u_t using a neural network u_t^θ , what loss function we are interested to train u_t^θ ?

- (A) $\mathcal{L}_{FM}(\theta) = \mathbb{E}_{t, X_t} \left[||u_t^\theta(X_t) - u_t(X_t)||^2 \right]$
- (B) $\mathcal{L}_{FM}(\theta) = \mathbb{E}_{t, X_t} \left[||u_t^\theta(X_t) + u_t(X_t)||^2 \right]$
- (C) $\mathcal{L}_{FM}(\theta) = \mathbb{E}_{t, X_t} \left[||u_t^\theta(X_t | X_{t-1}) - u_t(X_t)||^2 \right]$
- (D) $\mathcal{L}_{FM}(\theta) = \mathbb{E}_{t, X_t} \left[||u_t^\theta(X_t | X_{t+1}) - u_t(X_t)||^2 \right]$

Q10. When does SDE collapses to become ODE?

- (A) When the diffusion coefficient $\sigma_t = 0$.
- (B) When the diffusion coefficient $\sigma_t = \sigma$, a constant.
- (C) When the derivative of diffusion coefficient $\frac{d\sigma_t}{dt} = 0$.
- (D) When the derivative of diffusion coefficient σ_t cannot be determined.

Q11. Which one is true for Brownian motion:

- (A) Any sample at time t is Gaussian distributed.
- (B) Increment between two consecutive time is Gaussian distributed.
- (C) Increments are dependent on the previous increment.
- (D) In Brownian motion, the variance of the increment decreases with time.

Q12. Consider a random variable

$$\varepsilon \sim \mathcal{N}(0, \mathbf{I})$$

Imagine that $\mathbf{x}_0$ is the initial data distribution (the original data distribution).

Then, in the forward diffusion process, we have

$$\mathbf{x}_{t+1} = \sqrt{1 - \beta_t} \mathbf{x}_t + \sqrt{\beta_t} \varepsilon$$

Here, the recursive relationship collapses into a single step relative to $\mathbf{x}_0$:

$$\mathbf{x}_{t+1} = \sqrt{\bar{\alpha}_{t+1}} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_{t+1}} \varepsilon$$

Choose the correct option that provides relationship between α_i and β_i

- (A) $\alpha_i = \beta_i$, and $\bar{\alpha}_{t+1} = \sum_{i=0}^t \alpha_i$
- (B) $\alpha_i = 1 + \beta_i$, and $\bar{\alpha}_{t+1} = \sum_{i=0}^t \alpha_i$
- (C) $\alpha_i = 1 - \beta_i$, and $\bar{\alpha}_{t+1} = \prod_{i=0}^t \alpha_i$
- (D) $\alpha_i = 1 + \beta_i$, and $\bar{\alpha}_{t+1} = \prod_{i=0}^t \beta_i$

Q13. Referring to the previous problem, suppose somebody told you that initial data $\mathbf{x}_0$ has distribution $\mathcal{N}(m, s)$ where s is the variance and m is the mean.

In such a case, what is the mean/expectation of $\mathbf{x}_{t+1}$ in the forward diffusion process (Hint: use the property of expectations):

- (A) $\mathbb{E}[\mathbf{x}_{t+1}] = \sqrt{\bar{\alpha}_{t+1}} m$
- (B) $\mathbb{E}[\mathbf{x}_{t+1}] = \sqrt{\alpha_{t+1}} m$
- (C) $\mathbb{E}[\mathbf{x}_{t+1}] = \sqrt{\beta_{t+1}} m$
- (D) $\mathbb{E}[\mathbf{x}_{t+1}] = \sqrt{1 - \beta_{t+1}} m$

Q14. Consider a random variable

$$\varepsilon \sim \mathcal{N}(0, \mathbf{I})$$

then the new random variable

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon},$$

for constant $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ is a

- (A) A Gaussian distribution.
- (B) A Uniform distribution
- (C) An intractable distribution
- (D) A direc-delta distribution

Q15. If we have an input matrix as

$$\begin{bmatrix} 3 & 5 \\ 7 & 9 \end{bmatrix}$$

and we are looking to apply fractionally-strided convolution with a stride of 2, then how does the input changes before applying convolution using a kernel K .

(A)

$$\begin{bmatrix} 3 & 0 & 5 & 0 \\ 0 & 0 & 0 & 0 \\ 7 & 0 & 9 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \checkmark$$

(B)

$$\begin{bmatrix} 3 & 0 & 5 \\ 0 & 0 & 0 \\ 7 & 0 & 9 \end{bmatrix}$$

(C)

$$[3 \ 5 \ 7 \ 9]$$

(D)

$$\begin{bmatrix} 3 \\ 5 \\ 7 \\ 9 \end{bmatrix}$$

Q16. What does AutoEncoder do?

- (A) It creates a compressed representation of dataset.
- (B) It adds noise to dataset.
- (C) It can generate sythentic samples never seen before based on a training set.
- (D) It enhances image resolution.

Q17. Which one is the correct formula for KL Divergence from distribution q to distribution p ?

(A)

$$KL(q(\theta)||p(\theta|\mathcal{D})) = \int p(\theta) \log \frac{p(\theta|\mathcal{D})}{q(\theta)} d\theta$$

(B)

$$KL(q(\theta)||p(\theta|\mathcal{D})) = \int p(\theta) \log \frac{q(\theta)}{p(\theta|\mathcal{D})} d\theta$$

(C)

$$KL(q(\theta)||p(\theta|\mathcal{D})) = \int q(\theta) \log \frac{q(\theta)}{p(\theta|\mathcal{D})} d\theta$$

(D)

$$KL(q(\theta)||p(\theta|\mathcal{D})) = \int p(\theta) \log p(\theta|\mathcal{D})$$

Q18. Mean-field variational family assumes that

- (A) latent variables are independent of each other in the approximate posterior.
- (B) latent variables are orthogonal to each other in the approximate posteriror.
- (C) latent variables are linearly dependent on each oher in the approximate posterior.
- (D) latent variables have stochastic.

Q19. In the context of Variational Autoencoder which one is not true, assuming x as the input, z as the latent variable, and q as tractable approximation of the original distribution p ?

- (A) $q_w(z|x)$ maps x to a probability distribution over a latent space.
- (B) $p(z|x)$ is the intractable posterior, that is, it is very difficult to model.
- (C) $p(x)$ is known and you can always write easily.
- (D) $q_w(z|x)$ is the probabilistic encoder.

Q20. Consider the Bayesian framework:

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)p(\theta)}{p(\mathcal{D})}$$

with $p(\mathcal{D}) = \int p(\mathcal{D}|\theta)p(\theta)d\theta$.

Here, θ is a parameter or latent representation. $\mathcal{D}$ is dataset.

Out of all distributions, which is the most difficult to model?

- (A) $p(\theta|\mathcal{D})$
- (B) $p(\mathcal{D})$
- (C) $p(\theta)$
- (D) $p(\mathcal{D}|\theta)$

Q21. Consider the following code for the Generator:

```
class Generator(nn.Module):
    def __init__(self, in=28, height=28, width=28):
        super(Generator, self).__init__()
        self.in = in
        self.height, self.width = height, width

        self.model = nn.Sequential(
            nn.Linear(in, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 512),
            nn.LeakyReLU(0.2),
            nn.Linear(512, 1024),
            nn.LeakyReLU(0.2),
            nn.Linear(1024, width*height),
            nn.Tanh()
        )

    def forward(self, z):
        img = self.model(z)
        return img.view(img.size(0), 1,
                        self.height, self.width)
```

Based on the above code, what's the dimension of the noise that Generator uses in the default settings for mapping it to a possible image?

- (A) 256

- (B) 28
- (C) 512
- (D) 1024

Section 3 – – Short Answer

10 Points

Answer each prompt in the box. Write in complete sentences. 5 pts each.

3.1

Explain cycle-consistency as you learnt in Cycle GAN.

[5 pts]

Answer

3.2

Explain what are some issues encountered in GAN training.

[5 pts]

Answer

3.3

What do you understand by prior in Bayesian framework?

[5 pts]

Answer

3.4

KL Divergence integral equation is given by

$$KL(q(\theta)||p(\theta|\mathcal{D})) = \int q(\theta) \log \frac{q(\theta)}{p(\theta|\mathcal{D})} d\theta$$

Using mathematical formula of expectation from probability and statistics, derive its alternative form

$$KL(q(\theta)||p(\theta|\mathcal{D})) = \mathbb{E}_q[\log q(\theta)] - \mathbb{E}_q[\log p(\theta, \mathcal{D})] + \log p(\mathcal{D})$$

[5 pts]**Answer**

3.5

Derive the closed-form $KL(q||p)$ where:

$$q(z) = \mathcal{N}(z; m, s^2) = \frac{1}{\sqrt{2\pi s^2}} \exp\left(-\frac{(z-m)^2}{2s^2}\right)$$

$$p(z) = \mathcal{N}(z; \mu_0, \sigma_0^2) = \frac{1}{\sqrt{2\pi\sigma_0^2}} \exp\left(-\frac{(z-\mu_0)^2}{2\sigma_0^2}\right)$$

[5 pts]

Answer

3.6

For PINN Loss:

$$\mathcal{L}_{\text{total}}(\theta) = \underbrace{\lambda_d \frac{1}{N_d} \sum_{i=1}^{N_d} |u^\theta(x, t_i) - \hat{x}_i|^2}_{\text{Data Loss}} + \underbrace{\lambda_r \frac{1}{N_r} \sum_{j=1}^{N_r} \left| \frac{du^\theta(x, t_j)}{dt} \right|_{t_j}^2 + 2u^\theta(x, t_j)}_{\text{Residual Loss}}^2 + \underbrace{\lambda_0 |u^\theta(x, 0) - 1|^2}_{\text{Boundary Loss}}$$

explain each term in the above loss function.

[5 pts]

Answer

3.7

Explain why graph-like datasets are not compatabilite with multilayer-perceptron or fully connected artificial neural network , and how message-passing techniques create an embedding that is suitable to work with artificial neural network?

[5 pts]

Answer

3.8

What is the advantage of linear interpolant in Flow matching? Given the initial state as $\mathbf{x}_0$, and the final state as $\mathbf{x}_1$ with t as time, write the expression for the interpolant $\mathbf{x}_t$.

[5 pts]**Answer**

3.9

An MDP in the context of reinforcement learning is a 5-tuple model $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$. Explain each element of the 5-tuple model.

[5 pts]**Answer**

3.10

How does the reverse process in the Denoising Diffusion Model ensures that noisy version of the image gets mapped to the original version in the Markov chain process? Assume that we choose prior distribution as $p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$ and $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$. Further the original data is sampled from the real distribution $\mathbf{x}_0 \sim q(\mathbf{x})$.

[5 pts]

Answer

Section 4 – – KL Divergence

10 Points

Show all work for partial credit.

Compute KL divergence $KL(q||p)$ from q to p where $q = \mathcal{N}(2,1)$ and $p = \mathcal{N}(0,4)$.

Answer

Section 5 – – Reparametrization Trick

10 Points

The VAE encoder outputs $\boldsymbol{\mu} = [1.0, -0.5]$ and $\log \boldsymbol{\sigma}^2 = [0.0, -1.386]$. A noise sample is $\boldsymbol{\epsilon} = [0.8, -1.2]$. Compute the latent vector $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$.

Answer

Section 6 – – Code Inference and Understanding

(20 Points)

Consider a VAE network as shown below:

```

1  class VAE(nn.Module):
2      def __init__(self, input_dim=784, hidden_dim=400, latent_dim=20):
3          super(VAE, self).__init__()
4
5          # Encoder
6          self.encoder = nn.Sequential(
7              nn.Linear(input_dim, hidden_dim),
8              nn.ReLU(),
9          )
10         self.mu_layer = nn.Linear(hidden_dim, latent_dim)
11         self.logvar_layer = nn.Linear(hidden_dim, latent_dim)
12
13         # Decoder
14         self.decoder = nn.Sequential(
15             nn.Linear(latent_dim, hidden_dim),
16             nn.ReLU(),
17             nn.Linear(hidden_dim, input_dim),
18             nn.Sigmoid()          # Output in [0,1] for pixel values
19         )
20
21     def encode(self, x):
22         h = self.encoder(x)
23         mu = self.mu_layer(h)
24         logvar = self.logvar_layer(h)
25         return mu, logvar
26
27     def reparameterize(self, mu, logvar):
28         """Reparameterization trick: sample from N(mu, var) using N(0,1)."""
29         std = torch.exp(0.5 * logvar) # Standard deviation
30         eps = torch.randn_like(std)   # eps ~ N(0,1)
31         return mu + eps * std
32
33     def decode(self, z):
34         return self.decoder(z)
35
36     def forward(self, x):
37         # Flatten the image
38         x_flat = x.view(x.size(0), -1)
39         mu, logvar = self.encode(x_flat)
40         z = self.reparameterize(mu, logvar)
41         recon_x = self.decode(z)
42         return recon_x, mu, logvar

```

1. What happens when we do not use reparametrization trick?

2. Instead of variance, the network estimates, log of variances. Why do we do that?
3. Provide some understanding of what should be the loss function when training an instance VAE network defined above.
4. In the line `return mu + eps * std`, explain which variables carry gradients and which do not. Specifically: does `eps` have `requires_grad=True`? Through which variables does the gradient of the loss flow back to the encoder weights, and why does this make the sampling step differentiable?

Answer

Section 7 – 🎲 – Diffusion Process

(10 Points)

Consider a random variable

$$\varepsilon \sim \mathcal{N}(0, \mathbf{I})$$

Imagine that $\mathbf{x}_0$ is the initial data distribution (the original data distribution).

Then, in the forward diffusion process, we have

$$\mathbf{x}_{t+1} = \sqrt{1 - \beta_t} \mathbf{x}_t + \sqrt{\beta_t} \varepsilon$$

Here, the recursive relationship collapses into a single step relative to $\mathbf{x}_0$:

$$\mathbf{x}_{t+1} = \sqrt{\bar{\alpha}_{t+1}} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_{t+1}} \varepsilon$$

where we have the following relationship:

$$\alpha_i = 1 - \beta_i, \text{ and } \bar{\alpha}_{t+1} = \prod_{i=0}^t \alpha_i$$

Suppose somebody told you that initial data $\mathbf{x}_0$ has distribution $\mathcal{N}(m, s)$ where s is the variance and m is the mean.

1. Derive the mean $\mathbb{E}[\mathbf{x}_{t+1}]$ and the variance $\mathbb{V}(\mathbf{x}_{t+1})$ in the form of β_{t+1} , m and s .
2. Also provide the expression when $\beta_{t+1} = 0.1$ for all t .
3. Derive $q(\mathbf{x}_1 | \mathbf{x}_0)$.
4. In Diffusion process, in every step, we make sure that variance of the noisy data at time t doesn't explode. This is called variance preserving method. How does it guide us in choose what β_t we can use? How is it related to stochastic differential equation (SDE)?

In doing above derivation, please state what properties of expectations, and Gaussian distributions you are using.

Answer

Section 8 – – Diffusion Model: Code Inference and Understanding (10 Points)

8.1

```

1 def forward_diffuse(x0, t, alpha_bars):
2     sqrt_alpha_bar = alpha_bars[t] ** 0.5
3     sqrt_one_minus = (1 - alpha_bars[t]) ** 0.5
4     eps = torch.randn_like(x0)
5     xt = sqrt_alpha_bar * x0 + sqrt_one_minus * eps
6     return xt, eps

```

Identify the mathematical expression this function implements. Why is `eps` returned alongside `xt`? What would go wrong if you used `torch.randn(x0.shape)` instead of `torch.randn_like(x0)`?

8.2

Consider the code-snippet in the context of **Learning the Marginal Vector Field** for Flow Matching.

```

1 def sample_xt(z, eps, t):
2     return t * z + (1 - t) * eps
3
4 def conditional_vf(z, xt, t):
5     return (z - xt) / (1 - t + 1e-8)
6
7 def flow_matching_loss(model, z, device):
8     B = z.shape[0]
9     t = torch.rand(B, device=device)           # t ~ Uniform[0,1]
10    eps = torch.randn_like(z)
11    xt = sample_xt(z, eps, t.view(-1,1,1,1))
12    u_target = conditional_vf(z, xt, t.view(-1,1,1,1))
13    u_pred = model(xt, t)
14    return F.mse_loss(u_pred, u_target)

```

Explain the flow matching loss as implemented above.

8.3 Euler–Maruyama SDE Sampler

Consider the code-snippet in the context of Euler–Maruyama SDE Sampler:

```
1 def euler_maruyama(model, x, n_steps, sigma_schedule):
2     h = 1.0 / n_steps
3     for i in range(n_steps):
4         t = i / n_steps
5         u = model(x, t)
6         eps = torch.randn_like(x)
7         x = x + h * u + sigma_schedule[i] * (h ** 0.5) * eps
8     return x
```

Write down the Euler–Maruyama update rule’s mathematical expression for SDE sampling being implemented above.

Section 9 – – Graph Neural Network

(15 Points)

1. A graph has 4 nodes with undirected edges: (1,2), (1,3), (2,3), (3,4). Write the binary adjacency matrix A and degree matrix D .
2. Consider the code-snippet

```

1         from torch_geometric.data import Data
2     import torch
3
4     x = torch.tensor([[1, 0], [0, 1], [1, 1], [0, 0]], dtype=torch.float)
5     edge_index = torch.tensor([[0, 1, 1, 2, 2, 3],
6                               [1, 0, 2, 1, 3, 2]], dtype=torch.long)
7     data = Data(x=x, edge_index=edge_index)
8
9     print(data.num_nodes)
10    print(data.num_edges)
11    print(data.num_node_features)

```

What are the printed values of `num_nodes`, `num_edges`, and `num_node_features`? What graph does `edge_index` represent directed or undirected and why are edges listed twice?

```

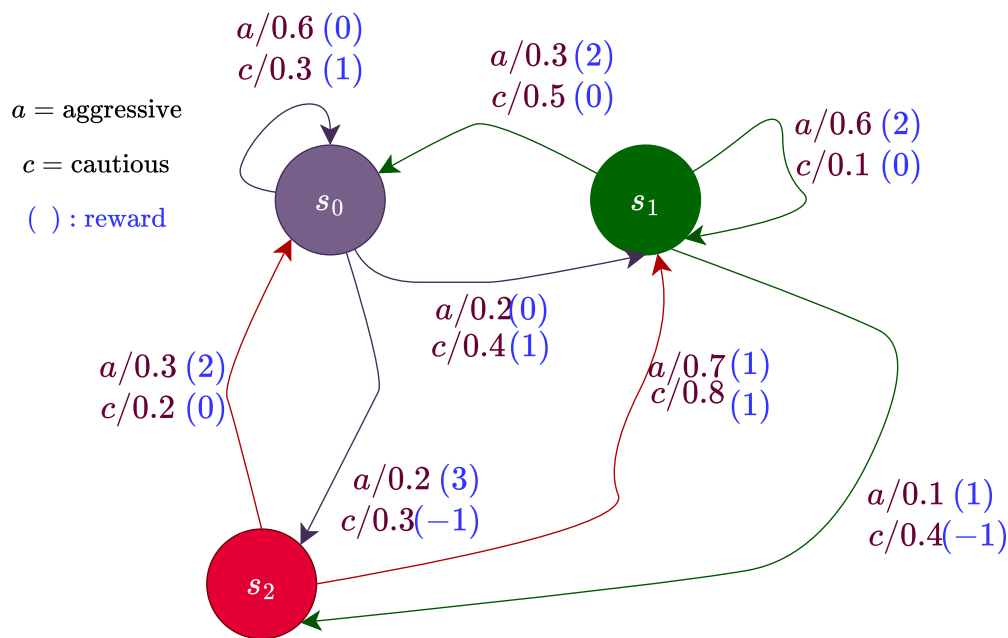
13.         from torch_geometric.nn import GCNConv
14     import torch.nn.functional as F
15
16     class GCN(torch.nn.Module):
17         def __init__(self, in_channels, hidden_channels, out_channels):
18             super().__init__()
19             self.conv1 = GCNConv(in_channels, hidden_channels)
20             self.conv2 = GCNConv(hidden_channels, out_channels)
21
22         def forward(self, x, edge_index):
23             x = self.conv1(x, edge_index)
24             x = F.relu(x)
25             x = F.dropout(x, p=0.5, training=self.training)
26             x = self.conv2(x, edge_index)
27             return x
28
29     model = GCN(2, 16, 3)

```

In the above code-snippet, how many GNN layers deep is this model, and what is the maximum hop distance each node's embedding incorporates?

Section 10 – – Markov Decision Process

(15 Points)



1. Expected immediate reward is given as

$$\bar{R}(s, a) = \sum_{s'} P(s'|s, a) R(s, a, s')$$

Calculate $\bar{R}(s_0, a)$ and $\bar{R}(s_0, c)$. Show your work.

(5 Points)

2. Based solely on expected immediate reward, which action (aggressive or cautious) is preferred in each state? Does the action stay consistent across all three states? (5 Points)
3. Suppose the agent follows the always aggressive policy π_a starting from s_1 . What is the expected reward after exactly one step? Repeat the same for always-cautious policy π_c . (5 Points)

Scratch page for calculation.

Scratch page for calculation.